Publicación y suscripción en aplicaciones Java

Librería para Kafka

- **Distribuida por Apache para su utilización desde publicadores y suscriptores.**

- **Requiere la dependencia Maven:**

```
<dependency>
   <groupId>org.apache.kafka</groupId>
   <artifactId>kafka-clients</artifactId>
   <version>3.5.1</version>
</dependency>
```

JAVADOC

https://javadoc.io/doc/org.apache.kafka/kafka-clients/latest/index.html

javadoc.io/doc/org.apache.kafka/kafka-clients/latest/index.html

org.apache.kafka kafka-clients 3.5.1

All Classes

Packages

org.apache.kafka.clients.admin
org.apache.kafka.clients.consumer
org.apache.kafka.clients.producer
org.apache.kafka.common
org.apache.kafka.common.acl

KafkaException
KafkaFuture
KafkaFuture.BaseFunction
KafkaFuture.BiConsumer
KafkaFuture.Function
KafkaMetric
KafkaMetricsContext
KafkaPrincipal
KafkaPrincipalBuilder
KafkaPrincipalSerde
KafkaProducer
KafkaStorageException
LeaderNotAvailableException
ListConsumerGroupOffsetsOptions
ListConsumerGroupOffsetsResult
ListConsumerGroupOffsetsSpec
ListConsumerGroupsOptions
ListConsumerGroupsResult
ListDeserializer
ListenerNotFoundException

org.apache.kafka.clients.producer

Class KafkaProducer<K,V>

java.lang.Object
org.apache.kafka.clients.producer.KafkaProducer<K,V>

All Implemented Interfaces:

Closeable, AutoCloseable, Producer<K,V>

```
public class KafkaProducer<K,V>
extends Object
implements Producer<K,V>
```

A Kafka client that publishes records to the Kafka cluster.

The producer is *thread safe* and sharing a single producer instance across threads will generally be faster than having multiple instances.

Here is a simple example of using the producer to send records with strings containing sequential numbers as the key/value pairs.

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("linger.ms", 1);
props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");
props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");
```

Envío de mensajes desde un productor

- **Creación de un productor. Se crea un objeto KafkaProducer con las propiedades de conexión a Kafka:**

KafkaProducer(Properties props)

- **Creación del mensaje. Se crea un ProducerRecord con los datos del mensaje a enviar**

ProducerRecord(String topic, V mensaje)

- **Envío del mensaje. A través del método *send()* de KafkaProducer se envía el mensaje:**

send(ProducerRecord record)

Antonio Martín Sierra

Recepción de mensajes en un consumidor

- **Creación de un consumidor. Se crea un objeto KafkaConsumer con las propiedades de conexión a Kafka:**

```
KafkaConsumer(Properties props)
```

- **Suscripción a tópico. A través del método *subscribe()* de KafkaConsumer se realiza la suscripción a tópico**

```
subscribe(Collection<String> topics)
```

- **Consumir mensajes. Mediante el método *poll()*, se consulta al tópico por la existencia de un mensaje, que es devuelto como un ConsumerRecord:**

```
ConsumerRecords<String,String> poll (Duration duracion)
```

Antonio Martín Sierra

Ejemplo productor

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("acks", "all");
props.put("key.serializer","org.apache.kafka.common.serialization.StringSerializer");
props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");
KafkaProducer<String, String> producer = new KafkaProducer<String, String>(props);
producer.send(new ProducerRecord<String, String>("mytopic1", "probando"));
```

Antonio Martin

Ejemplo consumidor

```
Properties props = new Properties();
//es necesario que los consumidores pertenezcan a un grupo. Si son del mismo
//grupo actúan como cola (solo lo consume uno), si son de distinto como
//tópico (los consumen todos)
props.put("group.id", "GroupA");
props.put("bootstrap.servers", "localhost:9092");
props.put("key.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
props.put("value.deserializer","org.apache.kafka.common.serialization.StringDeserializer");
KafkaConsumer<String, String> consumer = new KafkaConsumer<String, String>(props);
consumer.subscribe(Arrays.asList("mytopic1"));
while (true) {
   ConsumerRecords<String, String> records = consumer.poll(Duration.ofMillis(100));
   for (ConsumerRecord<String, String> record : records) {
       System.out.printf("offset = %d, key = %s, value = %s",
                                 record.offset(), record.key(), record.value()+"\n");
   }
 }
```